Deep Learning-Based Face Spoof Detection Using CNN

WING LEE*, San Jose State University, USA

This Project is to introduce and present a deep learning approach for face spoof detection using a Convolutional Neural Network (CNN). The system takes in video data by extracting its frames and then applying the CNN model and setting the threshold to classify inputs as real or fake. A controlled dataset with a balanced of real and fake samples was used during testing and when evaluating the testing we used the whole dataset. The model was evaluated using biometric performance metrics such as confusion matrix, False Acceptance Rate (FAR), False Rejection Rate (FRR), Reciver Operating Characterisitc (ROC) curve, and Equal Error Rate (EER). While the model recieved high accuracy results, the tradeoff between false acceptance and usability depended on the decision threshold. The optimal point was determined using the EER, finding the fair balancing point where FAR = FRR.

Additional Key Words and Phrases: face spoof detection, convolutional neural network, biometrics, deep learning, FAR, FRR, EER

ACM Reference Format:
Wing Lee. 2018. Deep Learning-Based Face Spoof Detection Using CNN. *ACM Trans. Graph.* 37, 4, Article 111 (August 2018), 4 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Facial recognition system in the modern world is widely used in biometric authentication, such as mobile devices, banking systems and for surveillance applications. These systems are prone to vulnerabilities such as spoofing attacks, where the attackers attempt to deceive the system by using fake inputs such as printed photos or fake replay videos or manipulated data.

Detecting spoofing attempts is important to make sure that there are no vulnerabilities and to make sure that it is secure. Traditional methods used more physical methods and are outdated, more recent methods using Artificial Intelligence deep learning have shown strong improvement in learning the different features from a set of data.

In this project we aim to make a convolutional neural network (CNN), using the FakeAVCeleb dataset. The CNN takes inputs and then learns automatically from the input frames and then classifies them as real or fake using binary classification where 0 is fake and 1 is real.

The model then is evaluated using biometric performance metrics such as False Acceptance Rate (FAR), False Rejection Rate (FRR), Receiver Operating Characteristic (ROC) curve, and Equal Error Rate (EER). The Results shows a tradeoff between accuracy, its security and its usability, based on the decision threshold.

Author's Contact Information: Wing Lee, wing.lee@sjsu.edu, San Jose State University, San Jose, California, USA.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM 1557-7368/2018/8-ART111
https://doi.org/XXXXXXX.XXXXXXX

2 Methodology Overview

This section shows the overall methodology I used for the face spoof detection. The program takes in video data from the FakeAVCeleb dataset and then processes it through several stages such as frame extraction, preprocesssing, CNN model and then final classification into biometric performance metrics.

Firstly, videos from the FakeAVCeleb dataset are split into training and testing sets. To improve learning and reduce bias, a random subset of training data was used, while the full test set is retained for evaluation. Frames are extracted from each video and then resized to a fixed resolution.

The processed frames are then sent to the CNN model in which it learns the different features through multiple convolutional pooling layers. After learning the features they are used to classify if its real or fake. The model outputs a probability score which is converted to binary prediction using the decision threshold.

Lastly, the program is evaluated using standard biometric performance metrics, including confusion matrix, FAR, FRR, ROC curve, and EER.

2.1 Model Architecture

This section describes the architecture of the convolutional neural network (CNN) used for face spoof detection. The model takes input frames of size 128×128 and processes them through multiple convolutional and pooling layers to extract important visual features.

The CNN has several convolutional layers with increasing numbers of filters, followed by max pooling layers to reduce spatial dimensions while keeping important information. These layers allow the model to learn hierarchical features such as edges, textures, and more complex patterns that distinguish real faces from spoofed inputs.

After feature extraction, the output is flattened and passed through fully connected layers, including a dropout layer to reduce overfitting. The final layer uses a sigmoid activation function to output a probability score between 0 and 1, representing the likelihood that the input frame is real or fake.

3 Modifications

This section presents the experimental results and findings of the proposed model. The dataset contains a total of 21,544 videos, which were split into 17,235 training videos and 4,309 testing videos. Due to class imbalance, only a balanced subset of 2,273 videos was used during training.

Each video was converted into multiple frames for processing. Approximately 5 frames were extracted per video, resulting in about 21,545 test samples. This explains why the number of evaluation samples is significantly larger than the number of videos.

The model was evaluated using accuracy, confusion matrix, False Acceptance Rate (FAR), and False Rejection Rate (FRR). Different decision thresholds were tested, including 0.5, 0.3, and the Equal Error Rate (EER) threshold. The EER threshold was found to provide the best balance between FAR and FRR.

4 CNN Architecture

This section describes the architecture of the convolutional neural network (CNN) used for face spoof detection.

The model takes input frames of size 128×128×3, representing RGB images. The first convolutional layer applies 32 filters with a 3×3 kernel and uses the ReLU activation function to detect basic features such as edges and textures. A max-pooling layer follows, which reduces the spatial dimensions of the feature maps while preserving the most important information.

The second convolutional layer increases the number of filters to 64, allowing the model to learn more complex patterns. This is again followed by a max-pooling layer to further reduce dimensionality and improve computational efficiency.

The third convolutional layer uses 128 filters, enabling the model to capture higher-level features such as facial structures and subtle artifacts that may indicate spoofing. Another max-pooling layer is applied after this stage.

Then after that it is flatten and uses relu activation function with a drop out of 0.5

Finally, the output layer consists of a single neuron with a sigmoid activation function, which produces a probability score between 0 and 1. This value represents the likelihood that the input frame is real, and is later converted into a binary classification using a decision threshold.

5 Binary cross-entropy

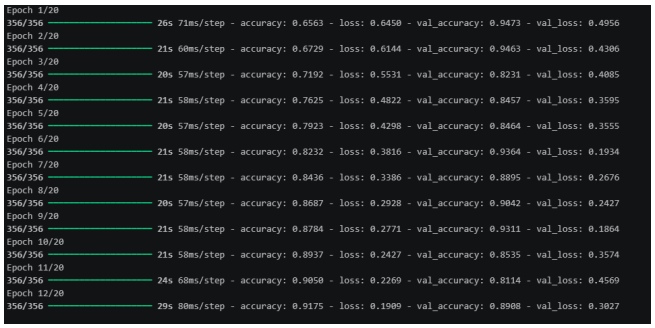
The model was made using the Adam optimizer which is an adaptive learning method that adjusts the learning rate during its training to help improve convergence. The loss function used is binary cross-entropy which is needed for a binary classification problem and it measures the difference between predicted probabilities and its actual labels. Accuracy was also used as an evaluation metric to measure the proportion of correct predictions during training phase.

6 EarlyStopping Method

To prevent overfitting, an early stopping method was implemented during training. The model was monitored using validation loss, and training was automatically stopped if the validation loss did not improve for three consecutive epochs, thus why we used patience = 3. This lets the program know to stop after it does not see further improvement. which means the CNN is not learning more as it is redundant.

Additionally, we take the best model weights after training to make sure the final model corresponds to the lowest validation loss observed during training.

As shown from Figure 1, the training and validation performance of the CNN model over multiple epochs. As training progresses, the training accuracy increases while the training loss decreases,



```
Epoch 1/20
356/356 ━━━━━━━━━━━━━━━━━━━━ 26s 71ms/step - accuracy: 0.6563 - loss: 0.6450 - val_accuracy: 0.9473 - val_loss: 0.4956
Epoch 2/20
356/356 ━━━━━━━━━━━━━━━━━━━━ 21s 60ms/step - accuracy: 0.6729 - loss: 0.6144 - val_accuracy: 0.9463 - val_loss: 0.4306
Epoch 3/20
356/356 ━━━━━━━━━━━━━━━━━━━━ 20s 57ms/step - accuracy: 0.7192 - loss: 0.5531 - val_accuracy: 0.8231 - val_loss: 0.4085
Epoch 4/20
356/356 ━━━━━━━━━━━━━━━━━━━━ 21s 58ms/step - accuracy: 0.7625 - loss: 0.4822 - val_accuracy: 0.8457 - val_loss: 0.3595
Epoch 5/20
356/356 ━━━━━━━━━━━━━━━━━━━━ 20s 57ms/step - accuracy: 0.7923 - loss: 0.4298 - val_accuracy: 0.8464 - val_loss: 0.3555
Epoch 6/20
356/356 ━━━━━━━━━━━━━━━━━━━━ 21s 58ms/step - accuracy: 0.8232 - loss: 0.3816 - val_accuracy: 0.9364 - val_loss: 0.1934
Epoch 7/20
356/356 ━━━━━━━━━━━━━━━━━━━━ 21s 58ms/step - accuracy: 0.8436 - loss: 0.3386 - val_accuracy: 0.8895 - val_loss: 0.2676
Epoch 8/20
356/356 ━━━━━━━━━━━━━━━━━━━━ 20s 57ms/step - accuracy: 0.8687 - loss: 0.2928 - val_accuracy: 0.9042 - val_loss: 0.2427
Epoch 9/20
356/356 ━━━━━━━━━━━━━━━━━━━━ 21s 58ms/step - accuracy: 0.8784 - loss: 0.2771 - val_accuracy: 0.9311 - val_loss: 0.1864
Epoch 10/20
356/356 ━━━━━━━━━━━━━━━━━━━━ 21s 58ms/step - accuracy: 0.8937 - loss: 0.2427 - val_accuracy: 0.8535 - val_loss: 0.3574
Epoch 11/20
356/356 ━━━━━━━━━━━━━━━━━━━━ 24s 68ms/step - accuracy: 0.9050 - loss: 0.2269 - val_accuracy: 0.8114 - val_loss: 0.4569
Epoch 12/20
356/356 ━━━━━━━━━━━━━━━━━━━━ 29s 80ms/step - accuracy: 0.9175 - loss: 0.1909 - val_accuracy: 0.8908 - val_loss: 0.3027
```

Fig. 1. Training and validation performance over epochs showing accuracy and loss trends.

showing that the model is learning meaningful patterns from the data.

Initially the validation accuracy is high but it fluctuates alot across epochs. The validation loss decreases in the early stages, this suggests that there is improvement, but later stages show it increases which is a sign of overfitting.

We gave it a patience of 3 because this allows the model to continue training for a short period despite temporary fluctuation making sure that the increase in validation loss wasn't just due to noise and the model has an opportunity to learn before stopping.

This behavior justifies the use of early stopping, as the model stops training once the validation loss no longer improves. By restoring the best weights, the final model corresponds to the point where validation performance is optimal rather than the final epoch.

7 Prediction

After training the model makes a score between 0 and 1 for each input frame. This score shows the model's confidence that the frame is either real or fake. Although this value might be misinterpreted as a probability of it being real or fake, it is not the result probability but rather a confidence score learned from the data.

To decide whether it is real or fake we convert the scores into binary classification, a decision threshold is made. We initially set the threshold to 0.5 because it is the median of 0 and 1. But later on we used a more accurate approach. We used the Equal Error Rate (EER) which gives us a balance between the FAR AND FRR. Frames with score above the threshold are classified as real, while scores below the classified is fake (spoofed).

Because the threshold is data-dependent, values that may appear low in absolute terms can still be classified as real if they exceed the selected threshold. This highlights that classification decisions are based on relative confidence rather than absolute probability values.

8 Visual Prediction Representation

To further analyze the performance of the model, a subset of test samples was randomly selected for visualization. Equal numbers of real and fake samples were chosen to ensure a balanced qualitative evaluation.

Initially I did random samples because I thought it would be more fair, but after running 20 simple samples I kept resulting with fake images so I had to work around it by forcing a fair equal amount of fake images and real images so I can visually see what was the likelihood of it predicting correctly.

For each selected frame, the predicted label, ground truth label, model confidence score, and corresponding video source are displayed.

The model does not achieve perfect accuracy and occasionally misclassifies samples, predicting fake as real and vice versa. However, it achieves an overall accuracy of approximately 80 percent, indicating that it performs well despite some errors.

There are tradeoffs between performance metrics. Adjusting the decision threshold affects both accuracy and the False Rejection Rate (FRR). A lower False Rejection Rate (FRR) indicates that fewer real samples are incorrectly rejected as fake, meaning the model is more effective at correctly identifying genuine inputs. This improves usability by reducing the likelihood of rejecting legitimate users.

Fig. 2. Example prediction from the FakeAVCeleb dataset showing the model output, predicted label, actual label, and confidence score.

From Figure 2, the probability score is 0.859 which is way higher than 0.2236 the decision threshold that was calculated from the (EER) after using the default threshold of 0.5.

9 Checking average correctness from samples

To further evaluate the model qualitatively, a subset of 20 samples (10 real and 10 fake) was randomly selected from the test dataset. For each sample, the predicted label was compared with the ground truth label to determine whether the prediction was correct or incorrect.

The model correctly classified 15 out of 20 samples, resulting in an accuracy of 75 percent for this subset. This evaluation provides an intuitive understanding of the model's behavior on individual examples.

However, this result is based on a small sample size and does not represent the overall performance of the model. The true performance is better reflected by the full test evaluation using metrics such as accuracy, (FAR), (FRR), and (EER).

Fig. 3. Example of a false positive prediction from the FakeAVCeleb dataset. The model incorrectly classifies a fake sample as real with a confidence score of 0.529, which exceeds the decision threshold. This type of error contributes to the False Acceptance Rate (FAR).

10 Confusion Matrix

The confusion matrix summarizes the classification performance of the model. It shows the number of correct and incorrect predictions for both real and fake classes. A large number of true negatives indicates that the model is effective at correctly identifying fake samples. However, the presence of false positives and false negatives shows that the model still makes errors in distinguishing between real and fake inputs.

10.1 Performance Metrics

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{1}$$

$$\text{FAR} = \frac{FP}{FP + TN} \tag{2}$$

$$\text{FRR} = \frac{FN}{FN + TP} \tag{3}$$

10.2 Results

$$\text{Accuracy} = \frac{879 + 15805}{15805 + 879 + 4605 + 256} = 0.774 \tag{4}$$

$$\text{FAR} = \frac{4605}{4605 + 15805} = 0.226 \tag{5}$$

$$\text{FRR} = \frac{256}{256 + 879} = 0.226 \tag{6}$$

11 Equal Error Rate (EER)

The Equal Error Rate (EER) is used to evaluate the balance between the (FAR) and the (FRR). This represent the point in which (FAR) and (FARR) are equal or closest to each other.

To determine the Equal Error Rate (EER), the Receiver Operating Characteristic (ROC) curve is generated using the predicted confidence scores. The ROC curve evaluates model performance across different decision thresholds by computing the False Positive Rate (FPR) and True Positive Rate (TPR). The False Negative Rate (FNR),

which is equivalent to the False Rejection Rate (FRR), is calculated as:

$$\text{FNR} = 1 - \text{TPR}.$$

The (EER) is then determined by finding the point where the difference between (FPR) and (FNR) is minimized. Here the (EER)

11.1 EER Formula

$$\text{EER} = FAR = FRR \tag{7}$$

$$\text{FAR} = FPR \tag{8}$$

$$\text{FRR} = FNR \tag{9}$$

11.2 ROC Curve

The Receiver Operating Characteristic (ROC) curve is used to evaluate the performance of the model across different decision thresholds. It plots the False Positive Rate (FPR), also known as the False Acceptance Rate (FAR), against the True Positive Rate (TPR).

Each point on the curve represents a different threshold used to classify the predicted probabilities. The curve illustrates the tradeoff between correctly identifying real samples and incorrectly accepting fake samples.

The Equal Error Rate (EER) is identified on the ROC curve as the point where the False Positive Rate (FPR) and False Negative Rate (FNR) are equal or closest.

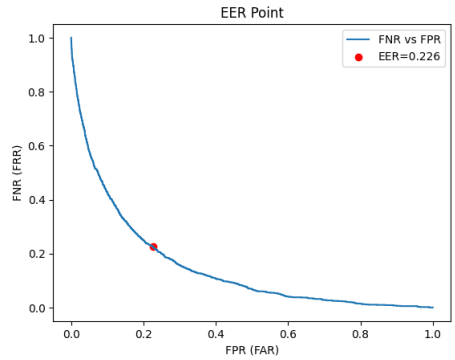


Fig. 4. ROC curve illustrating the relationship between False Positive Rate (FPR) and True Positive Rate (TPR). The highlighted point represents the Equal Error Rate (EER).

At this point we see that 22.6 percent of fake samples are incorrectly accepted and 22.6 percent of real samples are incorrectly rejected.